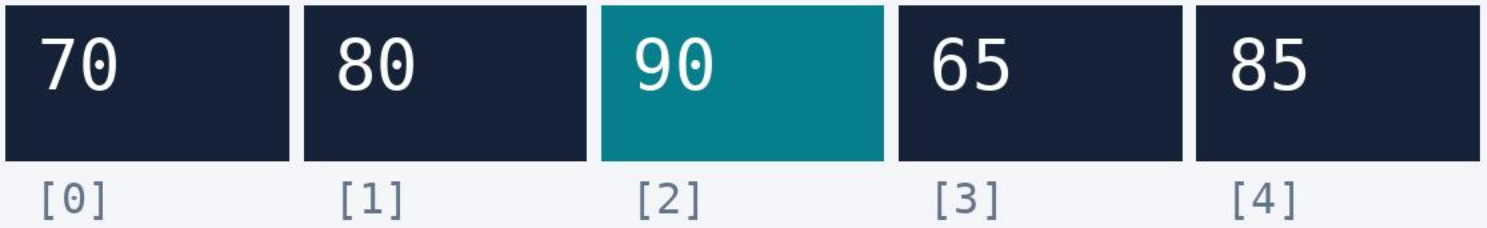
ARRAY IN C

จากข้อมูลหนึ่งค่า สู่การจัดการข้อมูลทั้งชุด

อ่านตำแหน่งให้เป็น จัดการข้อมูลให้ได้



01 Array datatype

02 1D arrays

03 Character arrays

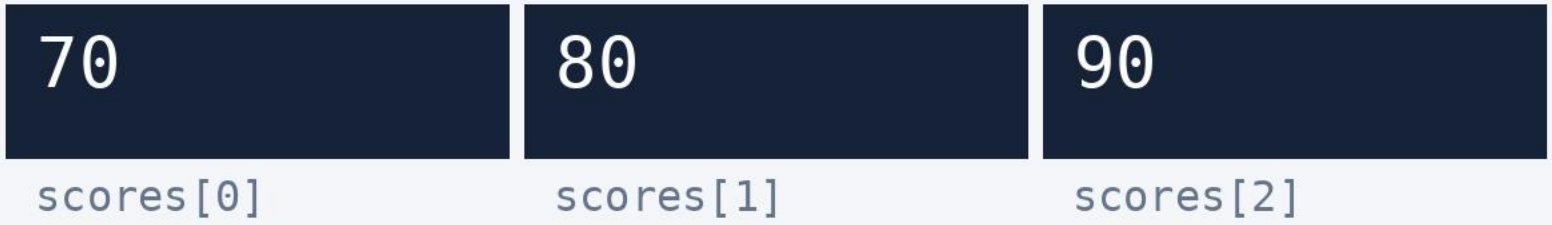
04 Multi-dimensional

Array คืออะไร?

สมาชิกชนิดเดียวกัน • อยู่ต่อเนื่องกัน • เข้าถึงผ่าน Index

C99 / EXAMPLE

```
int scores[3] = {70, 80, 90};
```



Index เริ่มที่ 0 > สมาชิกตัวแรกคือ scores[0]

ประกาศอาเรย์ 1 มิติ

ชนิดข้อมูล + ชื่ออาเรย์ + จำนวนสมาชิก

C99 / EXAMPLE

```
data_type array_name[size];

int scores[5];
float prices[10];
char letters[26];
```

int scores[5];

ชื่อ: scores
สมาชิก: 5 ตัว
ชนิดของสมาชิก: int
Index: 0 ถึง 4

อาเรย์ local ทั่วไปที่ยังไม่กำหนดค่า: ห้ามนำค่าที่ยังไม่กำหนดมาใช้

กำหนดค่าเริ่มต้นได้หลายรูปแบบ

ช่องที่ไม่ได้ระบุใน initializer จะถูกกำหนดเป็นศูนย์

`int a[4] = {1, 2, 3, 4};`	`1 2 3 4`
`int b[4] = {1, 2};`	`1 2 0 0`
`int c[4] = {0};`	`0 0 0 0`
`int d[4] = {0, 0, 0, 0};`	`0 0 0 0`
`int e[] = {7, 8, 9};`	`7 8 9 (3 items)`

int a[4] = {5}; ได้ 5, 0, 0, 0 — ถ้าต้องการทุกช่องเป็น 5 ให้ใช้ลูป

สร้างค่าเริ่มต้น != เปลี่ยนค่าภายหลัง

แยกการประกาศอาเรย์ออกจากการเขียนค่าลงในสมาชิก

C99 / EXAMPLE

```
int a[3] = {10, 20, 30};

a[0] = 99;
a[2] = 50;
```

ผลลัพธ์

a[0] > 99
a[1] > 20
a[2] > 50

เปลี่ยนได้ทีละสมาชิก

เขียนไม่ได้: int b[3] = 0; และ a = {1, 2, 3};

เข้าถึงสมาชิกอย่างถูกตำแหน่ง

อาเรย์ขนาด n ใช้ Index ตั้งแต่ 0 ถึง n - 1

C99 / EXAMPLE

```
int a[5] = {10, 20, 30, 40, 50};
printf("%d\n", a[0]);
printf("%d\n", a[4]);

a[1] = 99;
printf("%d\n", a[1]);
```

OUTPUT

```
10
50
99
```

a[5] และ a[-1] อยู่นอกขอบเขต > Undefined Behavior

ใช้ Loop เข้าถึงทุกสมาชิก

ให้ตัวแปร i วิ่งผ่าน Index ที่ใช้ได้

C99 / EXAMPLE

```c
int a[5] = {10, 20, 30, 40, 50};

for (int i = 0; i < 5; i++) {
    printf("a[%d] = %d\n", i, a[i]);
}
```

OUTPUT

```
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 40
a[4] = 50
```

ใช้ i < 5 ไม่ใช่ i <= 5

ตัวอย่าง: หาค่าสูงสุดและต่ำสุด

เริ่มจากสมาชิกตัวแรก แล้วตรวจสมาชิกที่เหลือ

C99 / EXAMPLE

```
int a[5] = {12, 7, 25, 3, 18};
int max = a[0], min = a[0];

for (int i = 1; i < 5; i++) {
    if (a[i] > max) max = a[i];
    if (a[i] < min) min = a[i];
}
printf("Max = %d\n", max);
printf("Min = %d\n", min);
```

OUTPUT

Max = 25
Min = 3

ตรวจครบทุกสมาชิก
เริ่มเปรียบเทียบที่ i = 1

ไม่ตั้ง max = 0 โดยไม่ดูข้อมูล เพราะข้อมูลอาจเป็นจำนวนลบทั้งหมด

ตัวอย่าง: ผลรวมและค่าเฉลี่ย

ค่าเฉลี่ย = ผลรวม ÷ จำนวนสมาชิก

C99 / EXAMPLE

```c
int scores[5] = {70, 80, 65, 90, 85};
double sum = 0;
for (int i = 0; i < 5; i++) {
    sum += scores[i];
}
double average = sum / 5;
printf("Sum = %.0f\n", sum);
printf("Average = %.2f\n", average);
```

OUTPUT

Sum = 390
Average = 78.00

ใช้ double เพื่อให้การหารเก็บทศนิยมได้

สลับค่า และคัดลอกข้อมูล

เก็บค่าเดิมไว้ก่อนเขียนทับด้วยตัวแปร temp

C99 / EXAMPLE

```
int a[3] = {10, 20, 30};
int temp = a[0];
a[0] = a[2];
a[2] = temp;

int b[3];
for (int i = 0; i < 3; i++) {
    b[i] = a[i];
}
```

หลังสลับค่า

a = {30, 20, 10}

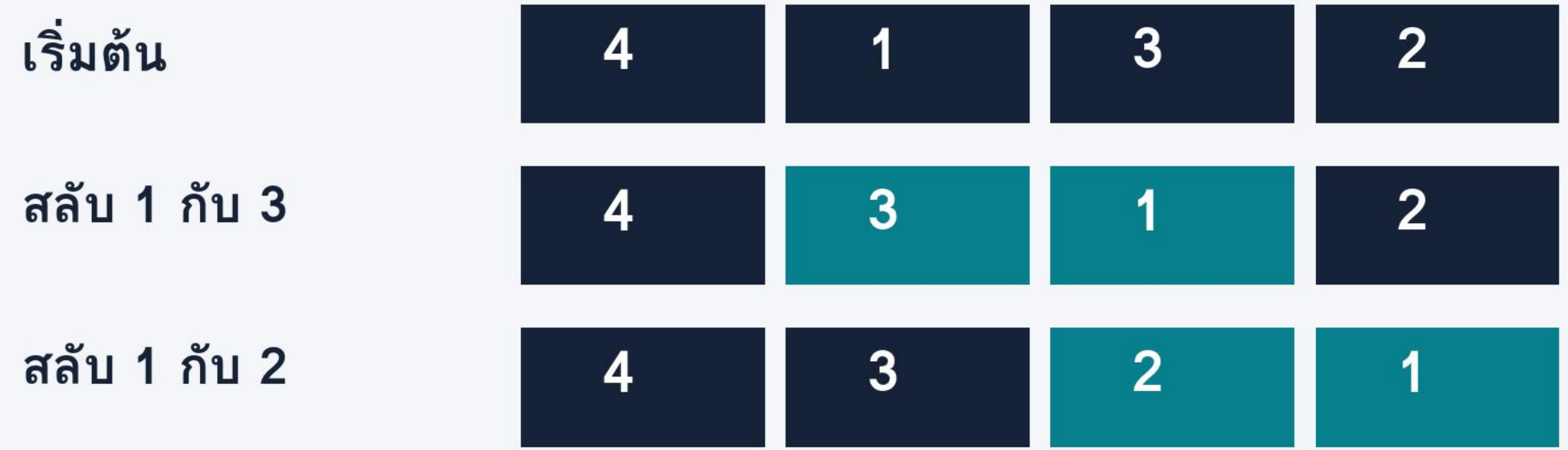
หลังคัดลอก
b = {30, 20, 10}

ปลายทางต้องมีที่พอ

คัดลอกทั้งอาเรย์ด้วย b = a; ไม่ได้ > คัดลอกทีละสมาชิกผ่านลูป

Bubble Sort: มาก > น้อย

เปรียบเทียบคู่ที่ติดกัน • ถ้าซ้ายน้อยกว่าขวา ให้สลับ



จบรอบแรก: ค่าน้อยที่สุดอยู่ขวาสุด • ข้อมูลบางชุดต้องทำต่ออีกหลายรอบ

Bubble Sort ด้วยลูปซ้อน

ลูปนอกนับรอบ • ลูปในเปรียบเทียบสมาชิก

C99 / EXAMPLE

```
int a[5] = {12, 7, 25, 3, 18};
int n = 5;
for (int pass = 0; pass < n - 1; pass++) {
  for (int j = 0; j < n - 1 - pass; j++) {
    if (a[j] < a[j + 1]) {
      int temp = a[j];
      a[j] = a[j + 1];
      a[j + 1] = temp;
    }
  }
}
for (int i = 0; i < n; i++)
  printf("%d ", a[i]);
```

OUTPUT

25 18 12 7 3

- pass ลดการตรวจส่วนท้ายที่เรียงแล้ว

เรียงน้อย > มาก:
เปลี่ยน < เป็น >

VLA: เลือกขนาดตอนรัน

Non-fixed size ในบทนี้: ขนาดมาจากค่าที่ทราบระหว่างรัน

C99 / EXAMPLE

```
int n;
if (scanf("%d", &n) != 1 ||
    n < 1 || n > 100) {
    return 1;
}
int a[n];
for (int i = 0; i < n; i++) {
    a[i] = 0;
}
printf("%d\n", a[n - 1]);
```

สิ่งที่ต้องจำ

ขนาดต้องเป็นบวก

สร้างแล้วขนาดคงเดิม แม้เปลี่ยนค่า n

C99: VLA ใช้ initializer ไม่ได้

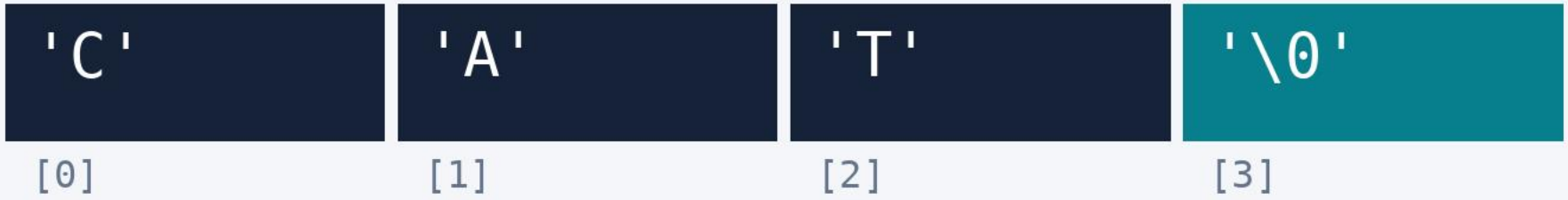
int a[n] = {0}; ใช้ไม่ได้กับ VLA ใน C99 > ใช้ลูปกำหนดค่า

String คืออาเรย์ของ char

String ต้องลงท้ายด้วย Null character: \0

C99 / EXAMPLE

```
char word[] = "CAT";
```



ข้อความยาว 3 ตัวอักษร แต่ใช้ 4 ช่อง • char array ที่ไม่มี \0 ไม่ใช่ String

ประกาศและกำหนดค่า String

เผื่อพื้นที่สำหรับ \0 เพิ่มอีกอย่างน้อย 1 ช่อง

C99 / EXAMPLE

```
char a[] = "Hello";
char b[6] = "Hello";
char c[10] = "Hello";
char d[] = {
  'H', 'e', 'l', 'l', 'o', '\0'
};
```

ขนาดอาเรย์

a, b, d > 6 ช่อง
c > 10 ช่อง

ข้อความยาว 5
แต่ต้องมี \0 ปิดท้าย

char bad[5] = "Hello"; ไม่มีที่สำหรับ \0 > ห้ามใช้เป็น String ตามปกติ

อ่านและแก้ไขตัวอักษร

ใช้ Index เช่นเดียวกับอาเรย์ 1 มิติ

C99 / EXAMPLE

```
char word[] = "CAT";
printf("%c\n", word[0]);

word[0] = 'B';
printf("%s\n", word);
```

OUTPUT

C
BAT

%c > ตัวอักษร
%s > String

'A' คือ char • "A" คือ String ที่มี 'A' และ '\0'

รับ String: หนึ่งคำ หรือทั้งบรรทัด

ทดลองสองตัวอย่างนี้แยกกัน

C99 / EXAMPLE

```c
char name[20];
if (scanf("%19s", name) == 1) {
  printf("%s\n", name);
}
```

scanf: อ่านหนึ่งคำ หยุดที่ช่องว่าง
%19s เผื่อช่องสุดท้ายสำหรับ \0

C99 / EXAMPLE

```c
char line[100];
if (fgets(line, sizeof line,
          stdin) != NULL) {
  printf("%s", line);
}
```

fgets: อ่านข้อความที่มีช่องว่างได้
อาจเก็บ \n ไว้ท้ายข้อความด้วย

scanf("%19s", name) ใช้ชื่อ char array โดยไม่เติม &

ฟังก์ชันพื้นฐานของ String

เพิ่ม #include <string.h> ก่อนเรียกใช้

`strlen(s)`	หาความยาว ไม่รวม \0	strlen("Cat") > 3
`strcpy(dst, src)`	คัดลอกข้อความ	strcpy(name, "Ann");
`strcat(dst, src)`	ต่อท้ายข้อความ	strcat(text, "!");
`strcmp(a, b)`	เปรียบเทียบข้อความ	ค่าเป็น 0 เมื่อเหมือนกัน

strcpy / strcat ต้องมีที่พอรวม \0 • strcmp คืนค่าลบ ศูนย์ หรือบวก

คัดลอก ต่อท้าย และเปรียบเทียบ

ตัวอย่างนี้ใช้อาเรย์ปลายทางขนาด 20 ช่อง

C99 / EXAMPLE

```
char text[20] = "Hello";
char copy[20];
strcat(text, " C");
strcpy(copy, text);
printf("%s\n", text);
printf("Length = %zu\n", strlen(text));
if (strcmp(text, copy) == 0) {
    printf("Same text\n");
}
```

OUTPUT

```
Hello C
Length = 7
Same text
```

strlen คืนค่า size_t
ใช้ %zu ใน C99

Reverse Word: กลับลำดับตัวอักษร

สลับตัวแรกกับตัวท้าย แล้วขยับเข้าหากึ่งกลาง

C99 / EXAMPLE

```
char word[] = "HELLO";
int length = (int)strlen(word);
for (int i = 0; i < length / 2; i++) {
    char temp = word[i];
    word[i] = word[length - 1 - i];
    word[length - 1 - i] = temp;
}
printf("%s\n", word);
```

ทีละขั้น

HELLO
v
OELLH
v
OLLEH

สลับเพียงครึ่งเดียว และไม่ย้าย \0 • ตัวอย่างนี้ใช้กับข้อความ ASCII

String Tips: จุดที่พลาดบ่อย

แยกความยาวข้อความออกจากขนาดพื้นที่ที่เตรียมไว้

C99 / EXAMPLE

```
char text[20] = "Hi";

printf("%zu\n", sizeof text); // 20
printf("%zu\n", strlen(text)); // 2
```

จำให้ขึ้นใจ

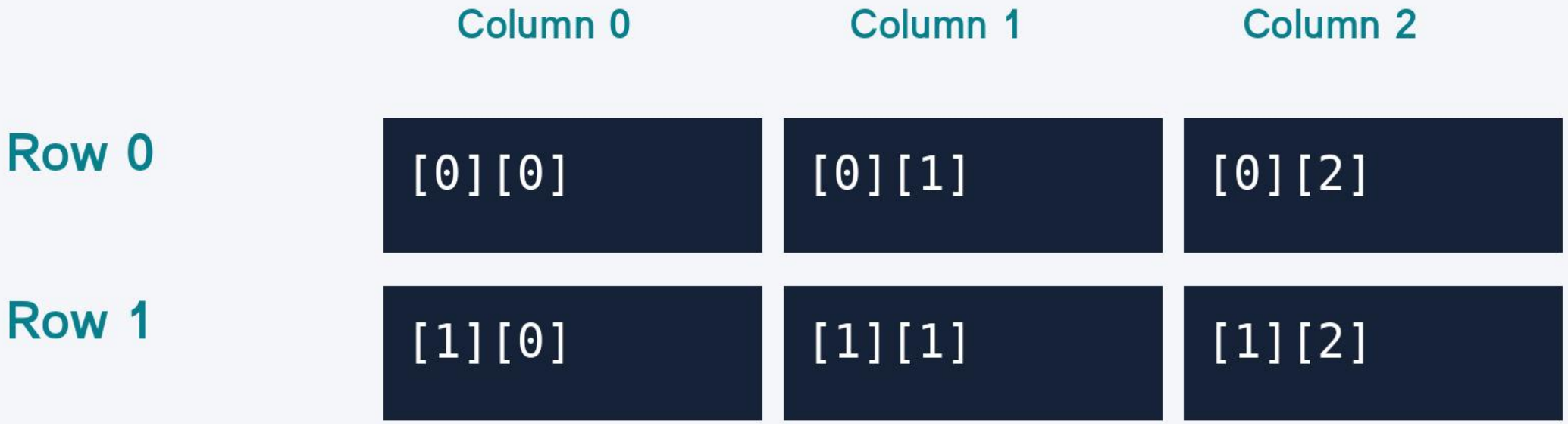
เผื่อ \0 เสมอ
เทียบข้อความ: strcmp
เปลี่ยนข้อความ: strcpy
รับข้อมูล: จำกัดความยาว

sizeof text: ขนาดอาเรย์นี้เป็นไบต์
strlen(text): ความยาวข้อความ ไม่รวม \0

scanf แล้วตามด้วย fgets: ระวัง \n ที่ค้างอยู่ในอินพุต

อาเรย์ 2 มิติ: แถว × คอลัมน์

int matrix[2][3]; > 2 แถว แถวละ 3 สมาชิก



เข้าถึงด้วย matrix[row][column] • Index ของทุกมิติเริ่มที่ 0

กำหนดค่าเริ่มต้นอาเรย์ 2 มิติ

ใช้วงเล็บปีกกาชั้นในเพื่อแยกข้อมูลแต่ละแถว

C99 / EXAMPLE

```
int a[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
int b[2][3] = {0};
int c[2][3] = {{1}, {2, 3}};
int d[][3] = {{1, 2, 3}, {4, 5, 6}};
```

ค่าที่ได้ใน c

1 0 0
2 3 0

b: ทุกสมาชิกเป็น 0

d: คอมไพเลอร์นับแถว

รูปแบบ d[][3] ละจำนวนแถวได้ แต่ยังต้องระบุจำนวนคอลัมน์

ใช้ลูปซ้อนกับอาเรย์ 2 มิติ

ลูปนอกเลือกแถว • ลูปในเลือกคอลัมน์

C99 / EXAMPLE

```
int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
a[1][2] = 99;
for (int row = 0; row < 2; row++) {
  for (int col = 0; col < 3; col++) {
    printf("%d ", a[row][col]);
  }
  printf("\n");
}
```

OUTPUT

```
1 2 3
4 5 99
```

หน่วยความจำเรียง
สมาชิกของแถว 0
ก่อนสมาชิกของแถว 1

อาเรย์ 3 มิติ: ชั้น × แถว × คอลัมน์

cube[2][2][3] มีสมาชิกทั้งหมด 2 × 2 × 3 = 12 ตัว

C99 / EXAMPLE

```
int cube[2][2][3] = {
  {{1, 2, 3}, {4, 5, 6}},
  {{7, 8, 9}, {10, 11, 12}}
};

printf("%d\n", cube[1][0][2]);
```

Layer 0

1	2	3
4	5	6

Layer 1

7	8	9
10	11	12

cube[1][0][2] = 9 > ชั้นที่ 1 / แถวที่ 0 / คอลัมน์ที่ 2

ลองทำด้วยตัวเอง

ใช้เฉพาะ Array, String, if และ loop ที่เรียนในบทนี้

01 / MAX • MIN • AVERAGE

สร้างอาเรย์จำนวนเต็ม 5 ค่า แล้วหาค่าสถิติ

02 / BUBBLE SORT

เรียงข้อมูลจากมากไปน้อย

03 / REVERSE WORD

รับคำอังกฤษไม่เกิน 19 ตัว แล้วกลับลำดับ

04 / 2D ARRAY

สร้างอาเรย์ 2 × 3 แล้วหาผลรวมแต่ละแถว

เช็กความเข้าใจ: int a[5] = {1}; แต่ละช่องมีค่าอะไร?